

DALIL (دليل) — Master Build Specification

For Claude Code. This file is the single source of truth for building the project. Drop it at the repository root; it also serves as `CLAUDE.md`. Read it fully before writing any code. Build in the phase order defined in §19 Build Roadmap. Do not skip the sovereignty, security, or compliance requirements — they are scored, not optional.

1. What we are building

Dalil is a production-grade, sovereign, multilingual AI travel companion for Algeria. It plans a personalized trip through conversation, guides the traveler on the ground via the phone camera, and lets them book hotels, Saharan tours, and artisan workshops — all on a fully Algerian-hosted data layer.

This is a competition entry for **IA Tour Algérie 2026** (Ministère du Tourisme et de l'Artisanat). It must read as a real product the State could adopt, not a demo. **Target: production-ready.**

The four capabilities

1. **Plan** — Conversational, multilingual trip planner (Darija / Arabic / Tamazight / French / English) that outputs a day-by-day itinerary on an interactive map, tuned to budget, mobility, interests, and family/halal constraints.
2. **See** — Point the camera at a monument or craft → instant recognition, the story narrated in the user's language, and a generative "golden-age" reconstruction for flagship heritage sites (drag-a-slider before/after).
3. **Speak** — Real-time assistance on the ground: Darija ↔ user-language translation, camera OCR for signs and menus, speech in/out.
4. **Book** — Hotels, Saharan tour operators, and artisan workshops in a single checkout, paid via an Algerian payment gateway.

Plus a **Sovereign Tourism Intelligence** admin dashboard (anonymized tourism-flow analytics) that serves the Ministry and demonstrates Axis 03.

The hero moment (the product must nail this)

A user stands before the ruins of Timgad, opens Dalil, points the camera. Dalil recognizes the site; a slider appears; dragging it rebuilds the Roman city as it stood in 200 AD while a voice narrates the story in Darija. The user then asks where to buy an authentic Berber rug nearby, and Dalil surfaces verified artisan workshops and books one. Every architectural decision should make this sequence flawless.

2. Competition constraints that drive engineering decisions

The judging grid (design backwards from it):

Criterion	Points	What it demands from the build
Innovation Technique	35	Real multimodal AI: agentic LLM, Darija fluency, vision recognition, generative reconstruction, RAG. Not a chatbot wrapper.
Impact Touristique	35	Genuinely removes the barriers to visiting Algeria (planning, on-ground friction, trust) and drives bookings.
Faisabilité & Scalabilité	20	Containerized, cheap per user, modular, deployable at scale on Algerian infra.
Qualité de la Présentation	10	Clean UX, fast, demo-able.

Article 10 (mandatory, and a differentiator): compliance with Algeria's personal-data law (Loi 18-07), a hardened architecture against cyber threats, and **bias mitigation** — models must be tested against discrimination by gender, region, or origin and must respect the **cultural and religious neutrality** of Algerian society. Treat these as acceptance criteria, not nice-to-haves.

Submission deadline: 19 August 2026, 23:59 Africa/Algiers. Build for that clock.

3. Non-negotiable principles

- Sovereignty first.** All persistent data lives in self-hosted PostgreSQL on Algerian infrastructure. **No Supabase, no Firebase, no managed US datastores.** The vector store (`pgvector`) and geospatial engine (`PostGIS`) are inside that same Postgres. Maps default to OpenStreetMap, not Google.
- Production-ready, not MVP.** Every feature ships with input validation, error handling, tests, logging, and access control. The repo must run end-to-end with one command (`docker compose up`) and deploy via CI/CD.
- Model-provider abstraction.** All AI calls go through a single internal interface so the underlying model can be swapped (frontier API today, self-hosted open model on Algerian hardware tomorrow). Never hardcode a provider in feature code.

4. **Cultural & religious neutrality by design.** System prompts and content guardrails enforce neutrality. No content that favors a region, gender, ethnicity, or religious posture. This is encoded in the AI layer and tested (§9, §10).
 5. **Cost-controlled AI.** LLM/vision endpoints are rate-limited, cached, and token-budgeted. Cost per user must stay low (feeds the Scalability score).
 6. **Multilingual + RTL from day one.** Arabic and Darija are RTL. i18n is not retrofitted later.
 7. **Accessibility.** WCAG 2.1 AA targets. Tourists include older users and users with disabilities.
-

4. Tech stack (decisive — use exactly this unless a hard blocker appears)

Runtime & language

- Node.js 22 LTS (note: Node 20 reaches the GitHub Actions deprecation window — pin Node 22), TypeScript 5.x (strict), pnpm.

Frontend / app framework

- Next.js 15 (App Router, Server Components, Server Actions), React 19.
- Tailwind CSS 4 + shadcn/ui (Radix primitives). Configured as an installable **PWA** (manifest + service worker) for camera access and an app-like feel on mobile.
- `next-intl` for i18n with full RTL support (ar, dz [Darija], ber [Tamazight], fr, en).

AI orchestration

- Vercel AI SDK (latest) for streaming chat, tool calling, and multi-provider abstraction. Wrap it in our own `lib/ai` provider interface (§9).

Database & data layer (the sovereign core)

- **PostgreSQL 16**, self-hosted via Docker.
- Extensions: `pgvector` (embeddings / RAG / image similarity), `PostGIS` (geospatial itineraries, nearby search), `pg_trgm` (fuzzy text), `uuid-oss`.
- **Drizzle ORM** + Drizzle Kit for schema, typed queries, and migrations. Use raw SQL migrations where extension-specific (vector/geometry columns, RLS policies).
- **Row-Level Security (RLS)** enforced at the Postgres layer for all user-owned data (this is the sovereign equivalent of Supabase RLS and is itself a talking point).

Auth

- Auth.js v5 (NextAuth) with the Drizzle adapter, JWT sessions, email/password + OTP. Fully self-hosted — no third-party auth provider.

Cache / queues / rate limiting

- Redis 7 for caching, rate limiting, and **BullMQ** background jobs (embedding generation, reconstruction-asset pipeline, booking confirmations).

Maps

- MapLibre GL JS + OpenStreetMap vector tiles (Protomaps/self-hostable). Architected so tiles can be self-hosted on Algerian infra. **Do not use Google Maps.**

Payments

- **Chargily Pay** (Algerian gateway: CIB / EDAHABIA). Wrap behind a `lib/payments` interface so additional providers can be added.

Optional inference microservice (Python)

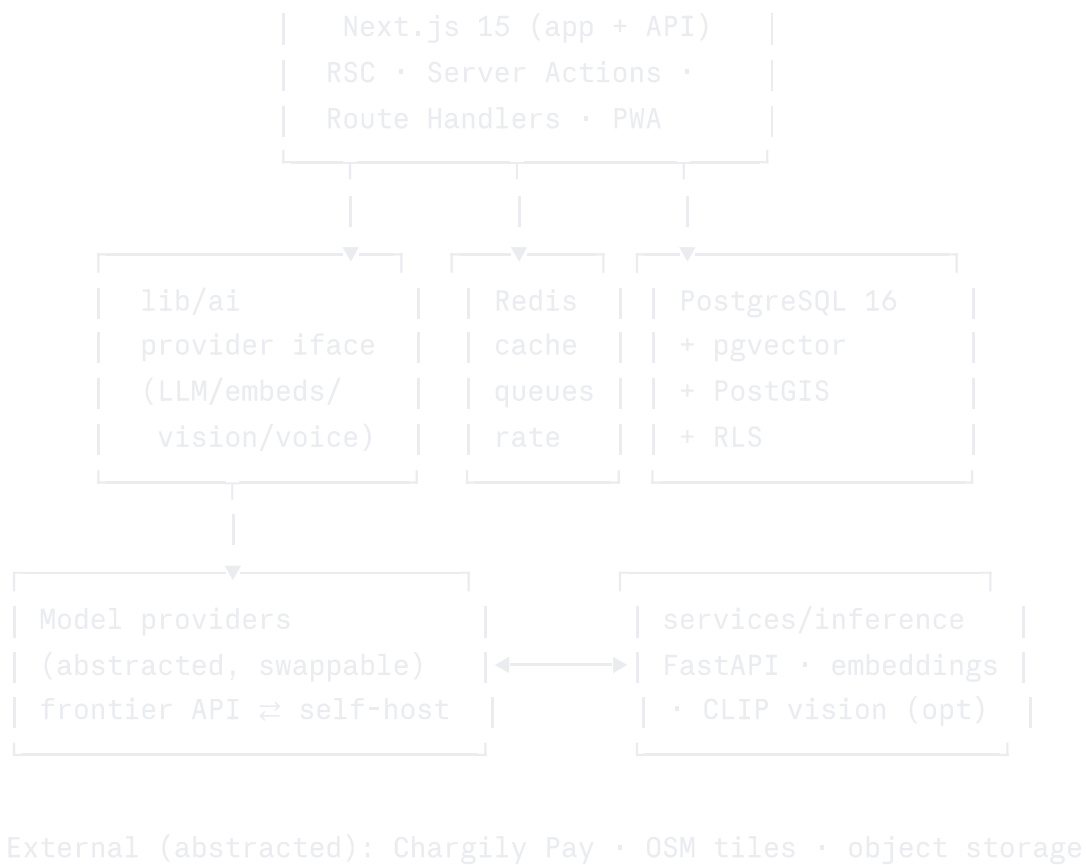
- A `services/inference` FastAPI app for sovereign open-model inference (text embeddings via a sentence-transformers model, image embeddings via a CLIP-class model for visual recognition). The Node app talks to it through the `lib/ai` interface. Default builds may call an embedding API; the spec must keep the door open to flip to this service without touching feature code.

Quality & ops

- Zod (validation), pino (structured logging), Vitest + Testing Library (unit/integration), Playwright (e2e), ESLint + Prettier, Husky + lint-staged.
- Docker + docker-compose (dev + prod), Nginx reverse proxy, GitHub Actions CI/CD.
- Sentry-compatible error tracking via a self-hostable target (e.g., GlitchTip); structured logs to stdout for the host's log stack.

5. System architecture





Data flow — "See" (visual recognition): camera frame → image embedding (inference service or API) → `pgvector` nearest-neighbour search over curated reference embeddings → matched `site`/`craft` → multimodal LLM narration in the user's language → reconstruction asset (if flagship site) streamed from storage.

Data flow — "Plan": user message → retrieve relevant chunks from `knowledge_chunks` via `pgvector` RAG → agentic LLM with tools (`search_sites`, `find_lodging`, `build_route` via PostGIS, `price_estimate`) → structured itinerary persisted to `itineraries` (+ PostGIS geometry) → rendered on MapLibre.

6. Data model (PostgreSQL schema)

Core tables (Drizzle schemas; vector/geometry columns via raw SQL migration). All user-owned tables carry `user_id` and an RLS policy.

Identity & profile

- `users`, `accounts`, `sessions`, `verification_tokens` (Auth.js).
- `profiles` — language preference, traveler type, accessibility needs, halal/family flags, home country (for diaspora segmentation).

Heritage & content (the sovereign knowledge base)

- `sites` — heritage/cultural/cultuel sites. Columns: name (multilingual JSONB), description (multilingual), `location geometry(Point,4326)`, region, category, opening info, practical tips, `is_flagship`.
- `site_media` — images/video, including reconstruction assets (`kind`: photo | reconstruction | video), storage key, era label.
- `site_reference_embeddings` — `embedding vector(N)` of curated reference images for visual recognition (pgvector, IVFFlat/HNSW index).
- `crafts` — artisanat taxonomy (multilingual), technique, origin region, authenticity markers.
- `artisans` — verified workshops: name, `location geometry(Point,4326)`, craft refs, verification status, contact.

Knowledge / RAG

- `knowledge_documents` — source docs (curated, Algerian-sourced) with provenance.
- `knowledge_chunks` — chunked text, multilingual, `embedding vector(N)` (pgvector), FK to document. HNSW index.

Trips

- `conversations`, `messages` — chat history per user (RLS).
- `itineraries` — title, dates, budget, `route geometry(LineString,4326)`, status, summary.
- `itinerary_items` — ordered stops: site/artisan/lodging/experience ref, day, time, `location geometry(Point,4326)`, notes.

Commerce

- `providers` — hotels, tour operators, agencies (incl. Touati-Travels-style entities). Verification status.
- `listings` — polymorphic inventory: `type` (hotel | sahara_tour | workshop_experience), provider FK, multilingual title/description, price, currency (DZD), availability JSONB, `location geometry`.
- `bookings`, `booking_items` — reservation records, traveler details, status state machine.
- `payments` — Chargily transaction records, status, idempotency key.

Intelligence & compliance

- `analytics_events` — anonymized interaction events (search intents, viewed sites, regions of interest) feeding the Tourism Intelligence dashboard. **No PII**. This is the

Axis 03 dataset.

- `audit_logs` — security/compliance trail (auth events, admin actions, data exports).
- `consent_records` — Loi 18-07 consent capture per user (purpose, timestamp, version).

Migration rules: every schema change is a committed Drizzle migration. Seed a curated demo dataset (see §18). Add `CHECK` constraints, FKs, and NOT NULLs liberally — this is production data.

7. Feature specs

7.1 Plan (Axis 02 — core)

- Streaming chat UI (shadcn) with language auto-detection and a manual language switcher (RTL-aware).
- Agentic LLM with tools: `search_sites`, `find_lodging`, `find_artisans`, `build_route` (PostGIS shortest sensible path between stops), `estimate_budget`.
- Output: a structured, editable day-by-day itinerary rendered on MapLibre, with drag-to-reorder, add/remove stops, and per-stop detail.
- Personalization: respects budget, pace, mobility, halal/family flags, and diaspora origin (e.g., surfacing a user's ancestral region).
- Persistence: itineraries saved per user (RLS), shareable via signed read-only links with correct OG/canonical tags (no localhost leakage in meta tags).

7.2 See (Axis 01 — hero)

- Camera capture (PWA, `getUserMedia`), client guidance overlay.
- Recognition: image embedding → `pgvector` similarity over `site_reference_embeddings` / craft references; confidence threshold with graceful "not sure" fallback.
- Narration: multimodal LLM produces the story in the user's language, grounded in the matched site's curated content (RAG, no hallucinated history).
- **Reconstruction:** for `is_flagship` sites, a pre-generated "golden-age" image/video asset rendered as a before/after slider over the live or captured frame. Assets produced offline by a generation pipeline (BullMQ job) and stored; **pre-rendered so the demo never breaks live**. Ship at least 3 flagship reconstructions (Timgad, Casbah d'Alger, Djémila or Tassili).
- Crafts: recognizing a craft returns technique, origin, authenticity markers, and nearby verified artisans.

7.3 Speak (Axis 02)

- Text + speech translation between Darija/Arabic/Tamazight/French/English, via the abstracted model layer.
- Darija handling: an Arabizi ↔ Arabic-script normalization step before/after model calls.
- Camera OCR mode for signs/menus → translation overlay.

7.4 Book (Axis 02 + Ministry's "réservation électronique")

- Unified search and detail pages for hotels, Saharan tours, artisan experiences.
- Cart → checkout → **Chargily Pay** (CIB/EDAHABIA), with idempotency, webhook confirmation, and a booking state machine (pending → confirmed → cancelled/completed).
- Email/SMS confirmation via background job. Provider-facing booking records.
- All inventory is real schema with seeded sample listings; integration points are documented for live provider onboarding.

7.5 Sovereign Tourism Intelligence (Axis 03 — admin)

- Authenticated admin dashboard for Ministry/operator roles.
 - Visualizations over `analytics_events`: demand heatmap by region (PostGIS), trending sites, interest by diaspora origin country, seasonality, and a **load-balancing view** that flags over-concentration and surfaces under-visited regions to redirect demand.
 - Strictly anonymized; the dashboard is the tangible proof of the data-sovereignty narrative.
 - Admin CMS to manage `sites`, `artisans`, `providers`, `listings`, and knowledge documents (production needs content management).
-

8. Internationalization & accessibility

- Locales: `ar`, `dz` (Darija), `ber` (Tamazight), `fr`, `en`. Default by Accept-Language with manual override.
- Full RTL layout for `ar`/`dz`: logical CSS properties, mirrored components, RTL-aware MapLibre controls.
- All user-facing strings in message catalogs; no hardcoded copy. Multilingual DB content stored as JSONB keyed by locale with graceful fallback.

- WCAG 2.1 AA: semantic HTML, focus management, color contrast, captions/transcripts for narrated audio, keyboard nav, reduced-motion support for the reconstruction slider.
-

9. AI layer design (`lib/ai`)

A single abstraction is mandatory. Feature code never imports a provider SDK directly.

Interfaces to implement:

- `chat(messages, tools, locale)` — streaming, tool-calling.
- `embedText(texts) -> vector[]`
- `embedImage(image) -> vector` (for visual recognition).
- `transcribe(audio)` / `synthesize(text, locale)` (Speak).
- `narrate(image, siteContext, locale)` (See).

Provider adapters:

- `providers/frontier.ts` — API-based (default for build speed).
- `providers/sovereign.ts` — talks to `services/inference` (self-hosted open models).
Switchable via env (`AI_PROVIDER=frontier|sovereign`).

RAG: retrieval over `knowledge_chunks` (pgvector, top-k + similarity threshold), citations passed to the model; the model must ground historical/cultural claims in retrieved content.

Guardrails (enforced here, tested in §10):

- System prompts enforce cultural/religious neutrality and prohibit regional/gender/origin favoritism.
 - Output filter for disallowed content.
 - Token budgets and response caching (Redis) on hot queries.
 - Per-user and per-IP rate limits on all AI endpoints.
-

10. Sovereignty, compliance & security (Article 10 — acceptance criteria)

Data residency & sovereignty

- PostgreSQL, Redis, object storage, and (if used) the inference service all self-hosted; deployment docs target an Algerian VPS/cloud. Document this explicitly in `/docs/sovereignty.md`.

Loi 18-07 (personal data)

- Consent capture (`consent_records`), a privacy policy page, data-export and account-deletion endpoints, data minimization (analytics carry no PII), and a documented retention policy.

Bias mitigation (tested)

- A `tests/ai/bias.spec.ts` suite probing recommendations and narration for region/gender/origin neutrality and cultural/religious neutrality, with documented methodology in `/docs/ethics.md`. CI runs it.

Cybersecurity hardening

- RLS on all user data; least-privilege DB roles. Zod validation on every input. Parameterized queries only. Security headers (CSP, HSTS, X-Frame-Options) via Nginx + Next. CSRF protection. Rate limiting. Secrets via env/secret store, never in the repo. Dependency and secret scanning in CI. Webhook signature verification (Chargily). Audit logging of sensitive actions.

11. Repository structure

```
dalil/
├─ apps/
│   └─ web/                # Next.js 15 app (app/, components/, lib/,
messages/)
│       └─ app/            # routes: (marketing), (app) plan|see|book,
admin, api/
│       └─ lib/ai/         # provider abstraction + adapters + RAG +
guardrails
│       └─ lib/db/         # drizzle schema, client, queries, migrations
│       └─ lib/payments/   # Chargily adapter behind interface
│       └─ lib/queue/      # BullMQ workers (embeddings, reconstruction,
notifications)
│       └─ messages/      # i18n catalogs: ar, dz, ber, fr, en
│       └─ tests/          # vitest + playwright
├─ services/
│   └─ inference/          # optional FastAPI: text/image embeddings, open
models
├─ infra/
│   └─ docker-compose.yml  # dev: postgres+pgvector+postgis, redis,
web, inference
```

```
| ├── docker-compose.prod.yml    # prod stack
| ├── nginx/                    # reverse proxy, TLS, headers, rate-limit
| └── postgres/                 # init scripts: extensions, roles
└── docs/
    ├── sovereignty.md ├── ethics.md ├── security.md ├── architecture.md
└── .github/workflows/         # ci.yml (lint, typecheck, test, build),
    deploy.yml
└── CLAUDE.md                  # this spec
└── README.md
```

12. Environment & configuration

Provide `.env.example` covering: `DATABASE_URL`, `REDIS_URL`, `AUTH_SECRET`, `AUTH_*`, `AI_PROVIDER`, model/embedding endpoints + keys (abstracted), `INFERENCE_URL`, `CHARGILY_*` (key + webhook secret), `MAP_TILES_URL`, `STORAGE_*`, `NEXT_PUBLIC_APP_URL`.
Validate env at boot with Zod; fail fast on missing required vars. **Never commit secrets.**

13. Local development

One command brings up the stack:

```
docker compose -f infra/docker-compose.yml up
```

Compose services: `postgres` (image with pgvector + PostGIS), `redis`, `web`, optional `inference`. App startup runs migrations (`drizzle-kit migrate`) and seeds (§18).
Document the full setup in README.

14. Deployment

- `docker-compose.prod.yml` + Nginx (TLS via Let's Encrypt, security headers, gzip/brotli, static cache, rate limiting) targeting an Algerian VPS.
 - Health-check endpoints (`/api/health`) for app, DB, Redis.
 - Backups: documented `pg_dump` cron + volume strategy.
 - Zero-downtime-ish deploy notes (migrate-then-switch).
-

15. CI/CD (GitHub Actions)

- `ci.yml`: install (pnpm) → typecheck → lint → unit/integration tests (spin a Postgres+pgvector service container) → build. Pin **Node 22** in `setup-node`; use current non-deprecated `actions/checkout` and `actions/setup-node`.
 - Run the bias suite (§10) in CI.
 - `deploy.yml`: build and push images, deploy on tag/release.
 - Secret scanning + dependency audit steps.
-

16. Testing strategy

- **Unit:** lib/ai guardrails, payment idempotency, RAG retrieval, route building (PostGIS), i18n fallbacks.
 - **Integration:** API route handlers against a test Postgres (pgvector/PostGIS), RLS enforcement (a user cannot read another user's itineraries), Chargily webhook handling with signature checks.
 - **E2E (Playwright):** plan a trip → see itinerary on map; recognize a seeded flagship site → reconstruction slider; book a listing → mock payment → confirmation; admin dashboard loads analytics.
 - **AI quality/bias:** §10 suite. Coverage gates in CI on critical paths.
-

17. Observability & logging

- pino structured JSON logs with request IDs; redact PII.
 - Error tracking to a self-hostable Sentry-compatible target.
 - Basic metrics: AI tokens/cost per request, cache hit rate, p95 latency on Plan/See, booking funnel events.
-

18. Seed data (for a complete, demo-able product)

Curate and seed (multilingual, Algerian-sourced, provenance recorded):

- **Flagship sites** with reconstruction assets: Timgad, Casbah d'Alger, Djémila, Tassili n'Ajjer, Ghardaïa (M'Zab), Tipaza, Constantine.
- **Crafts:** Berber rugs, M'Zab/Kabyle silver jewelry, pottery, copperwork, leather — with verified sample artisans.

- **Listings:** sample hotels (coast + Sahara), Saharan tour operators (Djanet/Tamanrasset), and artisan workshop experiences with realistic DZD pricing and availability.
- **Knowledge base:** curated docs behind the RAG layer for each flagship region, embedded into `knowledge_chunks`.

Seeds must be idempotent and runnable in CI/dev.

19. Build roadmap (execute in this order)

Phase 0 — Foundation. Monorepo + pnpm, Next.js 15 app, TypeScript strict, Tailwind 4 + shadcn, ESLint/Prettier/Husky. docker-compose with Postgres (pgvector+PostGIS) + Redis. Drizzle wired; first migration enables extensions. `.env.example` + Zod env validation. Health endpoint. **DoD:** `docker compose up` runs the app against a live DB; CI green (lint+typecheck+build).

Phase 1 — Data & auth. Full schema (§6) with migrations, RLS policies, and least-privilege roles. Auth.js v5 + Drizzle adapter (email/password + OTP). Seed framework + flagship/craft seeds. **DoD:** sign up/in works; RLS test proves cross-user isolation; seeds load.

Phase 2 — i18n + app shell. next-intl with 5 locales, RTL for ar/dz, language switcher, PWA manifest + service worker, base layout, navigation. **DoD:** locale switch flips copy and direction; app is installable.

Phase 3 — AI abstraction + RAG. `lib/ai` interfaces + frontier adapter + sovereign adapter stub. RAG over `knowledge_chunks` (pgvector). Guardrails (neutrality prompts, output filter, rate limit, cache). **DoD:** grounded Q&A with citations; guardrail unit tests pass.

Phase 4 — Plan. Streaming chat, agentic tools, PostGIS route building, itinerary persistence (RLS), MapLibre rendering, edit/reorder, shareable links with correct OG tags. **DoD:** end-to-end trip planned, saved, mapped, shared.

Phase 5 — See. Camera capture, image-embedding recognition over `pgvector`, grounded multilingual narration, reconstruction asset pipeline (BullMQ) + before/after slider, craft recognition → nearby artisans. **DoD:** seeded flagship site recognized; reconstruction slider works; craft lookup returns artisans.

Phase 6 — Book. Listings/search/detail, cart, checkout, Chargily integration (idempotency + webhook + signature), booking state machine, confirmation jobs. **DoD:** booking completes against mock/sandbox payment; webhook confirmed; record persisted.

Phase 7 — Speak. Translation (text/speech), Arabizi normalization, camera OCR overlay. **DoD:** Darija↔FR/EN round-trip; sign OCR translated.

Phase 9 — Compliance, hardening, polish. Loi 18-07 (consent, export, deletion, privacy page), bias suite in CI, security headers + CSP, full Playwright e2e, `/docs/*` (sovereignty, ethics, security, architecture), prod compose + Nginx + deploy workflow, backups. **DoD:** all gates green; one-command deploy documented; docs complete.

20. Coding standards

TypeScript strict everywhere; no `any` without justification. Zod-validate all external inputs. Server Actions/route handlers return typed results; errors handled at boundaries and logged. Small, focused modules; colocate tests. Conventional Commits. No secrets, no dead code, no TODOs left in shipped phases. Accessibility and i18n are part of "done," not follow-ups.

21. First actions for Claude Code

1. Confirm the stack in §4 and scaffold **Phase 0** exactly.
2. Generate `.env.example`, `infra/docker-compose.yml` (Postgres image with pgvector + PostGIS, Redis), and the first Drizzle migration enabling extensions.
3. Stand up the Next.js app with a health endpoint and green CI.
4. Stop at the Phase 0 DoD and report status before starting Phase 1.

Build deliberately, phase by phase, honoring every sovereignty, security, and compliance requirement. This is a product the Ministry should be able to deploy on Algerian soil — build it like one.